

Bug Tracking System

Database Design & SQL Report
MySQL | Database Systems Class Project

1. Database Overview

This database models a bug tracking domain: users with roles, projects with members, bugs with severity and status, comments, assignment history, and an audit log of field changes. Follows the course ERD with normalized tables and foreign key constraints.

Database Schema (8 Tables)

Table	Key Columns	Purpose
Role	role_id PK	User roles (admin, mgr, dev, tester)
User	user_id PK, role_id FK	Users linked to one role
Project	project_id PK, created_by FK	Projects with status & owner
ProjectMember	project_id + user_id PK	Many-to-many: users on projects
Bug	bug_id PK, project_id FK	Bugs: severity, priority, status
Comment	comment_id PK, bug_id FK	Comments on bugs
BugAssignment	assignment_id PK, bug_id FK	Assignment history per bug
BugHistory	history_id PK, bug_id FK	Audit log of field changes

Relationships

- User → Role (many-to-one)
- Project → User via created_by (many-to-one)
- Project ↔ User via ProjectMember (many-to-many)
- Project → Bug (one-to-many)
- Bug → User via reported_by & current_assignee_id
- Bug → Comment, BugAssignment, BugHistory (one-to-many each)

Sample Data (after seed)

Table	Rows
Role	4
User	7
Project	4
ProjectMember	13
Bug	14
Comment	13
BugAssignment	13
BugHistory	12

2. Five Complex SQL Queries

Each query combines multiple SQL techniques and returns analytical results from the seeded data.

Query 1: Project Bug Dashboard

Use Case: Dashboard summary per project: total, open, critical, closed bugs and avg priority.
Who: Project Manager / Team Lead | Concepts: LEFT JOIN, conditional SUM/CASE, GROUP BY, HAVING, AVG

```
SELECT p.project_id, p.name AS project, p.status AS project_status,
       COUNT(b.bug_id) AS total_bugs,
       SUM(CASE WHEN b.status IN ('open','in_progress','reopened') THEN 1 ELSE 0 END) AS open_bugs,
       SUM(CASE WHEN b.severity = 'critical' THEN 1 ELSE 0 END) AS critical_bugs,
       SUM(CASE WHEN b.status IN ('resolved','closed') THEN 1 ELSE 0 END) AS closed_bugs,
       ROUND(AVG(b.priority), 2) AS avg_priority
FROM Project p LEFT JOIN Bug b ON b.project_id = p.project_id
GROUP BY p.project_id, p.name, p.status HAVING total_bugs > 0
ORDER BY open_bugs DESC, critical_bugs DESC;
```

Project	Open	Critical	Closed	Avg Priority
Mobile App	3	1	1	2.25
API Platform	2	2	2	1.75
Website Redesign	2	1	2	2.50

Query 2: Developer Workload Report

Use Case: Shows each developer's assigned bugs, open count, critical open, and comment activity.
Who: Team Lead / Scrum Master | Concepts: Multi-table JOIN, COUNT(DISTINCT), conditional aggregation, correlated subquery

```
SELECT u.full_name, r.role_name,
       COUNT(DISTINCT b.bug_id) AS assigned_bugs,
       COUNT(DISTINCT CASE WHEN b.status IN ('open','in_progress','reopened') THEN b.bug_id END) AS open_bugs,
       COUNT(DISTINCT CASE WHEN b.severity='critical' AND b.status NOT IN ('resolved','closed') THEN b.bug_id
END) AS critical_open,
       (SELECT COUNT(*) FROM Comment c WHERE c.user_id = u.user_id) AS comments_written
FROM User u JOIN Role r ON r.role_id = u.role_id
LEFT JOIN Bug b ON b.current_assignee_id = u.user_id
WHERE r.role_name = 'developer'
GROUP BY u.user_id, u.full_name, r.role_name ORDER BY open_bugs DESC;
```

Developer	Assigned	Open	Critical Open	Comments
Bob Developer	7	4	2	4
Dave Developer	5	3	2	2

Query 3: Most Discussed Bugs (Activity Ranking)

Use Case: Ranks bugs by comment count + history change count to surface controversial or complex issues.

Who: QA Lead / Triage Team | Concepts: Multiple LEFT JOINS, COUNT(DISTINCT), GROUP BY, HAVING, ORDER BY, LIMIT

```
SELECT b.bug_id, p.name AS project, b.title, b.severity, b.status,
       COUNT(DISTINCT c.comment_id) AS comment_count,
       COUNT(DISTINCT h.history_id) AS history_count,
       COUNT(DISTINCT c.comment_id) + COUNT(DISTINCT h.history_id) AS total_activity
FROM Bug b JOIN Project p ON p.project_id = b.project_id
LEFT JOIN Comment c ON c.bug_id = b.bug_id
LEFT JOIN BugHistory h ON h.bug_id = b.bug_id
GROUP BY b.bug_id, p.name, b.title, b.severity, b.status
HAVING total_activity > 0 ORDER BY total_activity DESC LIMIT 10;
```

Bug	Project	Comments	History	Activity
500 on password reset	Website Redesign	3	2	5
Crash on startup	Mobile App	2	1	3
Offline sync fails	Mobile App	2	1	3

Query 4: Stale Open Bugs (Escalation Alert)

Use Case: Finds open bugs older than 30 days that are unassigned or high/critical severity for escalation.

Who: Release Manager / Support Lead | Concepts: DATEDIFF, COALESCE, multi-condition WHERE, LEFT JOIN

```
SELECT b.bug_id, p.name AS project, b.title, b.severity, b.priority, b.status,
       DATEDIFF(CURDATE(), DATE(b.created_at)) AS days_open,
       COALESCE(u.full_name, 'Unassigned') AS assignee
FROM Bug b JOIN Project p ON p.project_id = b.project_id
LEFT JOIN User u ON u.user_id = b.current_assignee_id
WHERE b.status IN ('open', 'in_progress', 'reopened')
      AND DATEDIFF(CURDATE(), DATE(b.created_at)) >= 30
      AND (b.current_assignee_id IS NULL OR b.severity IN ('critical', 'high'))
ORDER BY days_open DESC, b.priority ASC;
```

Bug	Project	Severity	Days Open	Assignee
Rate limit bypass	API Platform	critical	184	Dave Developer
JWT expiry not enforced	API Platform	critical	180	Bob Developer
500 on password reset	Website Redesign	critical	132	Bob Developer

Query 5: Resolution Rate by Severity

Use Case: Measures what % of bugs are resolved per severity level and avg days to resolve.

Who: Database Analyst / DBA | Concepts: GROUP BY, conditional SUM, percentage calc, AVG with DATEDIFF, FIELD ordering

```
SELECT b.severity,
       COUNT(*) AS total_bugs,
       SUM(CASE WHEN b.status IN ('resolved','closed') THEN 1 ELSE 0 END) AS resolved_bugs,
       ROUND(100 * SUM(CASE WHEN b.status IN ('resolved','closed') THEN 1 ELSE 0 END)
             / COUNT(*), 1) AS resolution_pct,
       ROUND(AVG(CASE WHEN b.resolved_at IS NOT NULL
                       THEN DATEDIFF(b.resolved_at, b.created_at) END), 1) AS avg_days_to_resolve
FROM Bug b GROUP BY b.severity
ORDER BY FIELD(b.severity, 'critical','high','medium','low');
```

Severity	Total	Resolved	Resolution %	Avg Days to Resolve
critical	4	0	0.0%	—
high	3	2	66.7%	26.0
medium	4	2	50.0%	18.0
low	3	1	33.3%	14.0

3. Conclusion

This project demonstrates relational modeling with 8 normalized tables, foreign key integrity, and ENUM constraints for controlled values (severity, status, roles). The five queries show how standard SQL — joins, GROUP BY, conditional aggregation, subqueries, and date functions — can extract meaningful analytical reports from a real-world schema without relying on application code.